by Kunal Sir

Abstraction (OOP's) in Java:

1. This is one of the major pillars of Oop's concept.

2. Without in detailed knowledge, we can apply the functionality in work with their essential knowledge that is called as abstraction.

3. In-detail knowledge means method internal code.

4. Essential knowledge means method name their parameter, return type and purpose.

5. The purpose of abstraction is to reduce the complexity.

6. We can achieve abstraction by the two ways –
    - **Interface**
    - **Abstract Class**

Abstraction by an example: -

Let's take the ATM machine as a real-time example.

We all use an ATM machine for cash withdrawal, money transfer, retrieve min-statement, etc. in our daily life.

But we don't know internally what things are happening inside ATM machine when you insert an ATM card for performing any kind of operation.

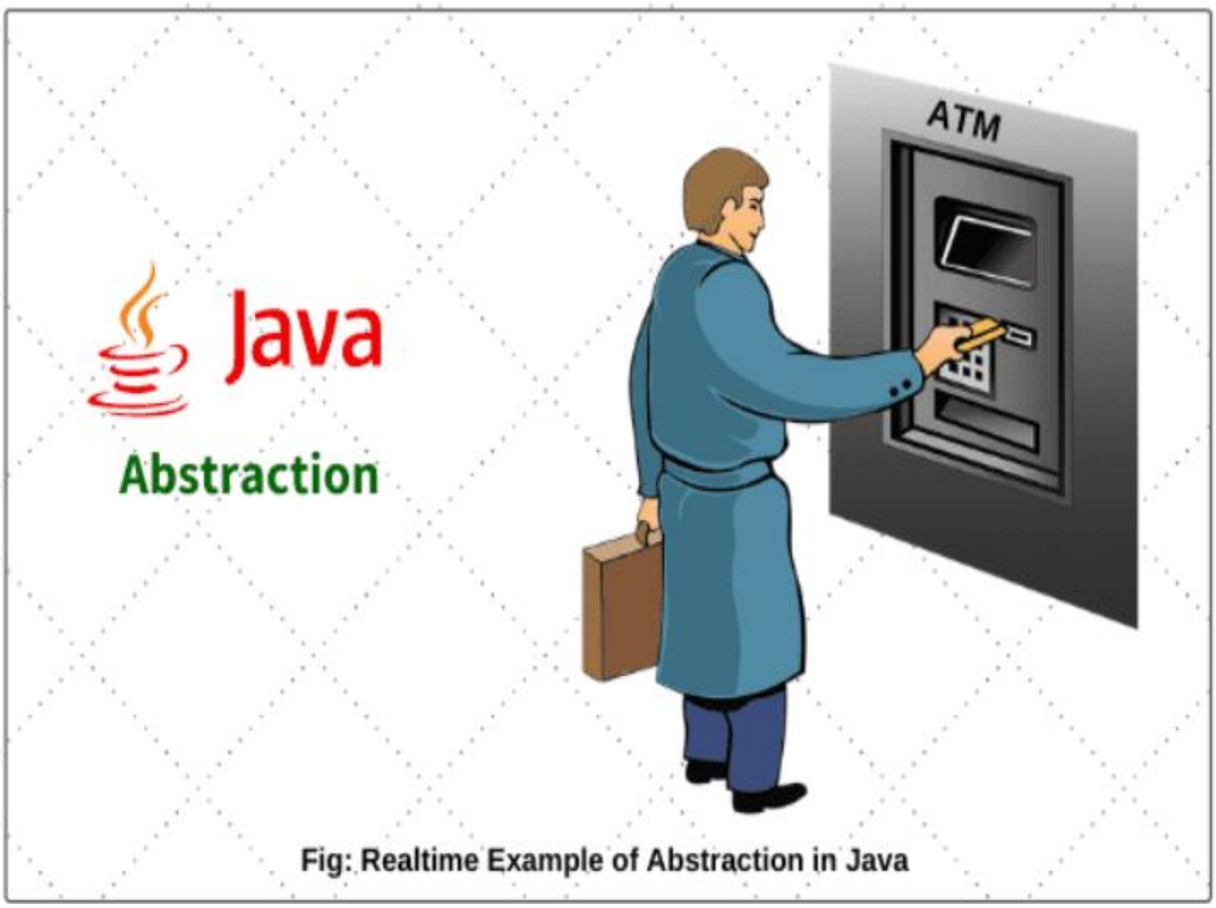


diagram of abstraction

by **Kunal Sir**

Interface in Abstraction:

1. Interface is the set of rules and guidelines.

2. In other words, you can say that interfaces can have abstract methods and variables. It cannot have a method with body.

Example: - Interface is just like RBI. RBI provides rules for all banks just like that Interface provides rules for all implemented class.

Rules for Interface:

- By using "interface" keyword we can designed interface in java.
- Interface variable we need to initialize compulsory.
- Interface variable is by default public, static and final.
- Till jdk1.7 in interface all the methods are abstract only.
- In interface abstract method it means there is no body of that method.

- Abstract method is only ended with semicolon.
- Abstract method by default public and abstract keyword.
- We cannot create an object of the interface.
- We can create only object reference of the interface by using their implemented class.

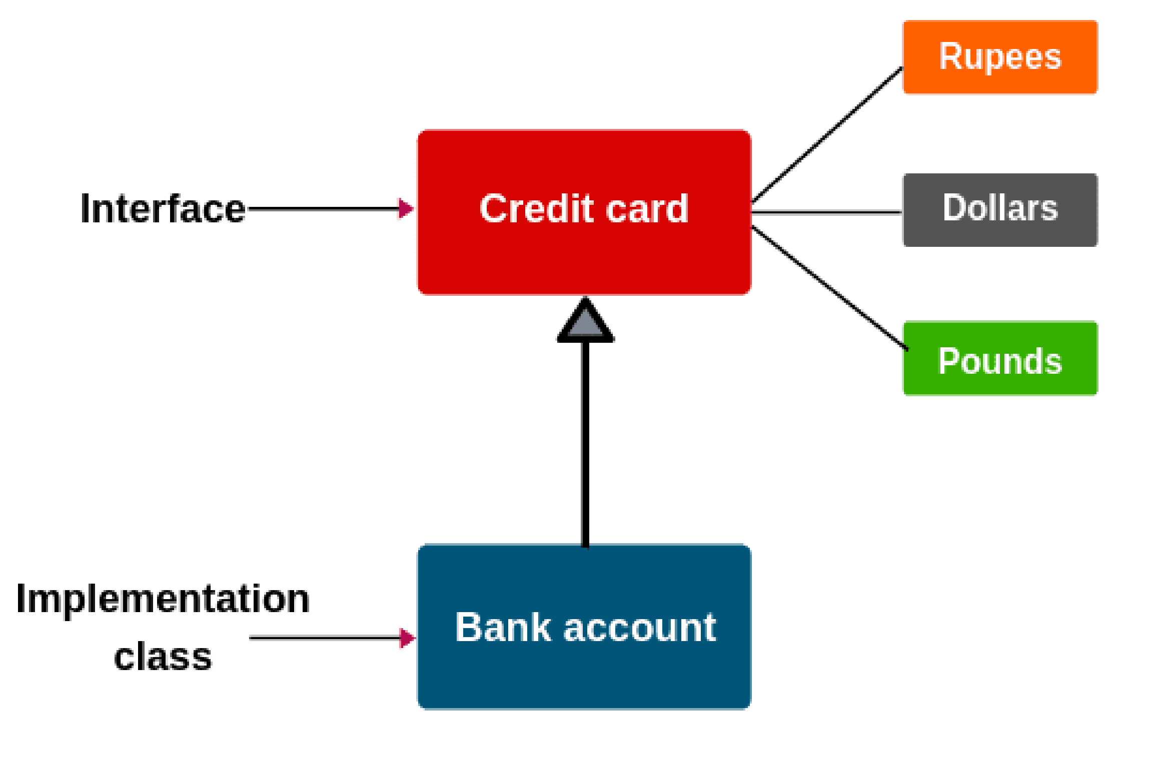
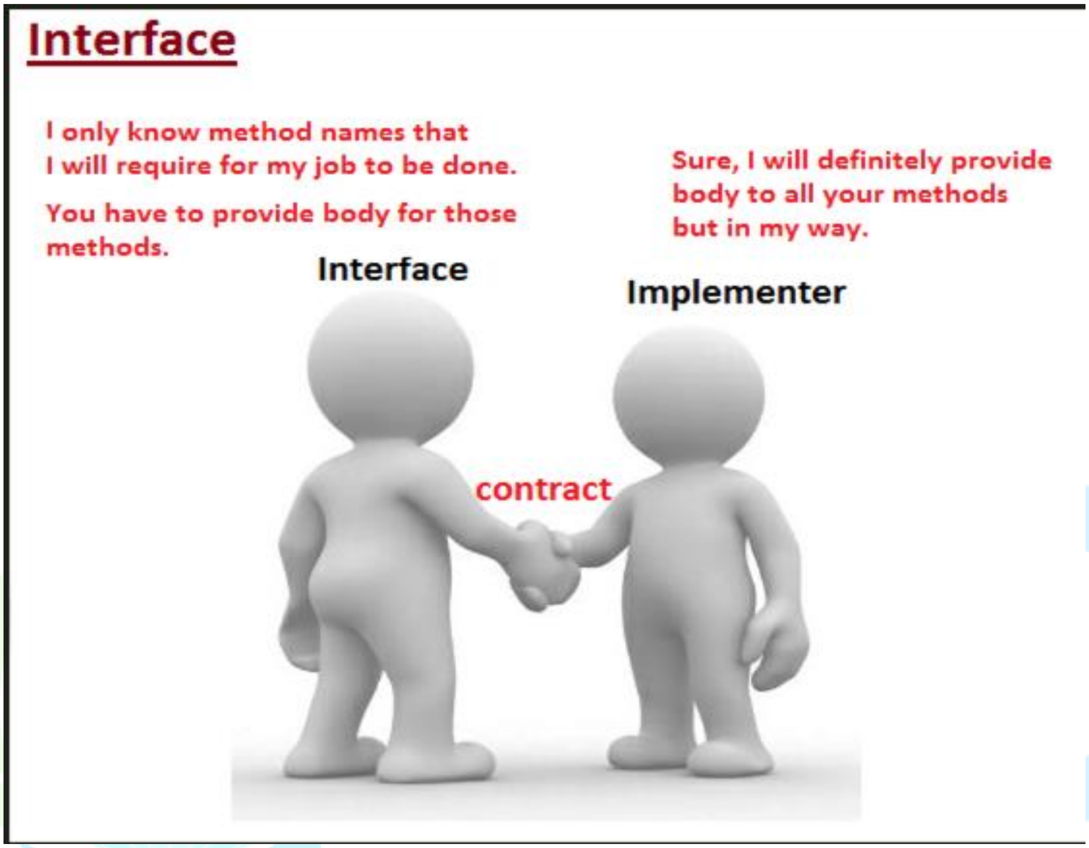


Fig: Interface and Implementation class

diagram of interface

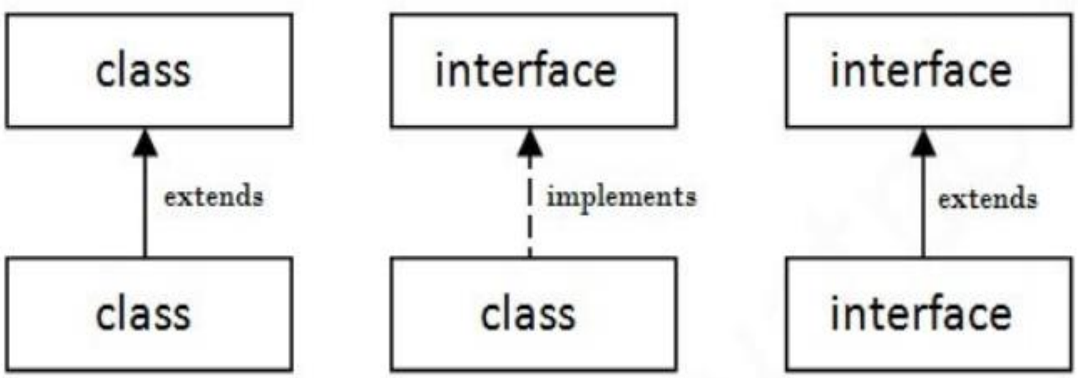


Syntax for Java Interface:

```
public interface InterfaceName {
    //declare variable by default static final
    //declare method that abstract by default
}
```

The Relationship Between Classes and Interfaces:

As shown in the figure given below, a class extends another class, an interface extends another interface, but a class implements an interface.



Note: - Important point to be remember-

- Java is supported the multiple inheritance by using interface because there is no ambiguity with interface.
- Class implements the interface property by using implements keyword.

by Kunal Sir

Why the interface does not contain any Constructor?

Since all data members are static and final, it means they all are class variable or I can say they are not an instance or object variable because of static keyword.

And if you remember the whole purpose of a constructor is to initialize the instance variable of the class.

Hence, no instance variables are available in Interface, so there is no meaning of constructor presence inside Interface.

Thus, the interface does not contain any Constructor.

What is the Marker Interface in java?

Marker interface in java interfaces with no field methods or, in simple words empty interface in java is called as a Marker Interface.

Marker interface is also known as Tag interface.

Example: - Serializable, Cloneable etc.

by Kunal Sir

What should we need of Marker Interface?

The purpose is to provide run-time type information about objects, which allows the compiler and jvm to have more information about the object.

Marker interface is for decision making jvm will take decision.

Understanding Program:

```
public interface I
{
    int x=20;
    void m1();
    void m2();
 }

 public class A implements I
{
    @override
    public void m1()
   {
       System.out.println("m1=A");
   }
   @override
```

Stop, Near, 1st Floor, Above Rupam Sweets/ Priyanka Collections Building Vikas Mitra Mandal Chowk Road, Karve Nagar, Pune, Maharashtra 411052, Mobile No.- 8888022204

CJC
Complete Java Classes

by Kunal Sir

```
   public void m2()
   {
        System.out.println("m2=A");
   }
   public void m3()
  {
       System.out.println("m3=A");
  }
}

public class Test
{
       public static void main(String[]args)
      {
            System.out.println(I.x);
            I i=new A();
              i.m1();
              i.m2();
              i.m3(); // incorrect
            System.out.println(i.x);
            A a=new A();
               a.m1();
               a.m2();
               a.m3();
      }
}
```